

UNIVERSIDAD PRIVADA DOMINGO SAVIO

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

ASIGNATURA: INGENIERÍA DE SOFTWARE I

ACTIVIDAD 03 — ÁLBUM DE MODELOS UML (ENTREGABLE GRUPAL)

CRITERIO DE VERIFICACIÓN #2 (30 PUNTOS) — GATE 3 DEL SDLC

"YAPU": DESCRIPCIÓN DEL DISEÑO DE SOFTWARE (IEEE STD 1016-2009 SDD)

**MAPA INTEGRAL DE MODELOS ARQUITECTÓNICOS, ESTRUCTURALES (UML),
ENTIDAD-RELACIÓN (ERD) Y DE COMPORTAMIENTO**

EQUIPO DE DESARROLLO (AUTORES):

Emmanuel Ponce Quiroga (Líder Técnico & Gobernanza de IA)

Jhoel Álvaro Cruz Zurita (Arquitectura VPS, Persistencia & Múltiples DBs)

Luis Mario Rocha Vela (Aseguramiento de Calidad & Estándares APA/IEEE)

DOCENTES DE CÁTEDRA:

Ing. Jimmy Nataniel Requena / Ing. Fernando Pardo

STAKEHOLDER PEDAGÓGICA:

Lic. María Elena Quispe Mamani (Docente Titular de Lengua Quechua — U.E. Simón Bolívar,
Sucre)

Santa Cruz de la Sierra / Sucre, Bolivia

Septiembre de 2026

RESUMEN EJECUTIVO

El presente documento técnico constituye el Álbum de Modelos UML y la Descripción del Diseño de Software (Software Design Description - SDD) para el proyecto YAPU (Sembradío en lengua quechua), desarrollado en estricto cumplimiento de los estándares internacionales IEEE Std 1016-2009 e IEEE Std 830-1998, conforme a las directrices del Gate 3 del ciclo de vida de desarrollo de software (SDLC) de la Universidad Privada Domingo Savio (UPDS). El proyecto YAPU es una Plataforma Web Progresiva (PWA) de alta resiliencia diseñada para la revitalización y enseñanza de la lengua quechua (variante sureña Collao/Chuquisaca), orientada a operar en comunidades urbanas y rurales con conectividad degradada (2G/3G) y smartphones de gama de entrada con Android 8.0+.

Este compendio traduce formalmente los 10 Requisitos Funcionales (RF-001 al RF-010), los Requisitos No Funcionales (RNF-001 al RNF-005) y los Requisitos de Sostenibilidad (RS-001 al RS-004) mediante 12 modelos visuales interconectados en Mermaid.js. Destacan: (1) una Arquitectura de 4 Capas basada en Astro SSG y Service Workers con consumo de RAM menor a 150 MB; (2) un Diagrama de Clases orientado a objetos con Inversión de Dependencias (DIP) y composición estricta; (3) una solución estructural multibase de datos que modela los schemas locales de IndexedDB (ObjectStores e índices) y los schemas documentales remotos de Cloud Firestore mediante Data Mappers; (4) un Motor de Inteligencia Artificial Determinista en TypeScript con coste cero (\$0) en APIs y ausencia total de alucinaciones lingüísticas; (5) un protocolo de curaduría comunitaria bajo el principio de los cuatro ojos (`autor_id != validador_id`); y (6) una Matriz de Auditoría Ética de IA que registra el control humano ejercido sobre las decisiones arquitectónicas del sistema.

CAPÍTULO I: INTRODUCCIÓN Y MARCO NORMATIVO

1.1 Propósito y Alcance del Documento (IEEE Std 1016-2009)

El estándar IEEE Std 1016-2009 (Standard for Information Technology — Systems Design — Software Design Descriptions) define la estructura formal para representar las vistas de diseño de un sistema de software. Este documento traduce la Especificación de Requisitos de Software (SRS) del Bloque 2 en especificaciones de diseño modulares, auditables y verificables mediante modelos UML y esquemas de datos.

1.2 El Proyecto YAPU y el Contexto Territorial

YAPU es una solución sociotécnica que responde a la alarmante pérdida de transmisión intergeneracional del quechua sureño en el departamento de Chuquisaca y zonas periurbanas de Sucre. En cumplimiento de la Constitución Política del Estado (Art. 5) y la Ley de Educación Avelino Siñani - Elizardo Pérez, el sistema proporciona una experiencia de aprendizaje gamificada cubriendo los primeros 10 niveles correspondientes al nivel A1 básico certificable.

1.3 Estructura de Vistas Arquitectónicas

Para garantizar una cobertura integral de los requerimientos, el Álbum de Modelos UML se organiza en cuatro vistas fundamentales:

- Vista Arquitectónica y Física: Topología de 4 capas, componentes y protocolos de transporte (Diagrama 1).
- Vista Estructural Orientada a Objetos: Clases del dominio pedagógico y clases de persistencia multibase de datos (Diagramas 2 y 3).
- Vista de Persistencia de Datos: Diagrama Entidad-Relación (ERD) y taxonomía de relaciones fuertes y débiles (Diagrama 4).
- Vista Funcional y de Comportamiento: Casos de uso general y específicos, diagramas de interacción (secuencia), máquinas de estado finito y diagramas de actividad (Diagramas 5 al 9).

CAPÍTULO II: VISTA ARQUITECTÓNICA Y FÍSICA

2.1 Arquitectura General del Sistema (Topología de 4 Capas)

El sistema se estructura en cuatro capas claramente desacopladas: Capa de Cliente (PWA Astro), Capa de Red Criptográfica, Capa de Servicios en la Nube (Google Firebase) y Capa de Infraestructura Autogestionada (VPS Linux UPDS con Nginx). En este diseño, la comunicación cliente-servidor se realiza mediante llamadas HTTP estándar sobre TLS 1.3. La aplicación no incluye rombos de decisión condicional en la red, ya que la mediación entre la memoria local y el tráfico de internet es gestionada de manera transparente por el Service Worker.

Código Mermaid del Diagrama 1: Arquitectura General

```
graph TB
    subgraph CLIENT_TIER [Capa de Cliente: PWA Ligera < 150MB RAM]
        UI[Interfaz Astro SSG + Islands]
        SW[Service Worker & Workbox]
        CACHE_API[(Cache Storage API)]
        IDB_LOCAL[(IndexedDB Local)]
        CORE_AI[Motor IA Determinista en TS]
    end
    subgraph CLOUD_TIER [Servicios Cloud: Google Firebase]
        FAUTH[Firebase Auth JWT]
        FSTORE[Cloud Firestore NoSQL]
        RULES[firestore.rules]
    end
    subgraph VPS_TIER [VPS Linux UPDS]
        NGINX[Reverse Proxy Nginx HTTP/2]
        ASSETS[Repositorio Assets WebP < 100KB]
        OPEN_DATA[Exportador Corpus CSV/JSON]
    end
    UI --> SW
    SW --> CACHE_API
    SW --> IDB_LOCAL
    UI --> CORE_AI
    SW -->|HTTPS TLS 1.3| FAUTH
    SW -->|HTTPS TLS 1.3 / gRPC| FSTORE
    SW -->|HTTPS TLS 1.3| NGINX
    IDB_LOCAL -. ->|Background Sync API| FSTORE
```

Tabla 1: Especificación de Componentes y Protocolos de Red

Componente	Capa / Tecnología	Justificación Técnica de Ingeniería
------------	-------------------	-------------------------------------

Astro SSG + Islands	Cliente / Astro 4.x	Renderizado estático con cero JS por defecto. Consumo RAM < 150MB (RS-002) y FCP < 1.8s (RNF-001).
Service Worker & Workbox	Cliente / Web Workers	Intercepta peticiones HTTP salientes. Aplica CacheFirst a medios y Background Sync a notas.
IndexedDB	Cliente / Almacén NoSQL	Persistencia transaccional no volátil en el navegador móvil para evaluaciones no sincronizadas.
Cloud Firestore	Cloud / Google BaaS	Base documental NoSQL multirregión con capacidad para absorber 500 sesiones concurrentes (RNF-005).
Nginx HTTP/2 VPS	VPS / Debian 12 UPDS	Reverse proxy de activos WebP comprimidos (< 100KB) y endpoint de datos abiertos (RS-004).

CAPÍTULO III: VISTA ESTRUCTURAL ORIENTADA A OBJETOS (DOMINIO PEDAGÓGICO)

3.1 Diagrama de Clases del Dominio Pedagógico y Motor IA (Diagrama 2)

El modelado de clases del núcleo de YAPU aplica rigurosamente los principios SOLID, en particular el Principio de Inversión de Dependencias (DIP) y el Principio de Responsabilidad Única (SRP). La clase `EvaluationController` interactúa exclusivamente con la interfaz abstracta `IEvaluationEngine`, lo que permite intercambiar el motor morfológico o inyectar objetos simulados (Mocks) en pruebas automatizadas sin tocar la interfaz gráfica.

Código Mermaid del Diagrama 2: Clases del Dominio

```
classDiagram
    class EvaluationController {
        -IEvaluationEngine evaluationEngine
        -IOfflineFirstRepository repository
        -StudentSession activeSession
        +initExam(levelId: int) void
        +submitAnswer(qId: String, opt: String) void
        +finalizeExam() EvaluationResult
    }
    class Evaluation {
        -String evaluationId
        -String studentId
        -int scorePercentage
        +calculateScore() int
        +isPassed() boolean
    }
    class Question {
        -String questionId
        -List~String~ distractors
        -boolean isCorrect
        +evaluateAnswer(selected: String) boolean
    }
    class IEvaluationEngine {
        <<interface>>
        +generateAssessment(levelId, corpus) List~Question~
        +validateAnswerDeterministically(qId, sel) boolean
    }
    class DeterministicAIEngine {
        -int PASSING_THRESHOLD = 70
        +generateAssessment(levelId, corpus) List~Question~
    }
    EvaluationController ..> IEvaluationEngine : DIP
    IEvaluationEngine <|.. DeterministicAIEngine : Realización
```

```
Evaluation "1" *-- "10" Question : Composición Estricta
EvaluationController o-- "1" StudentSession : Agregación
```

3.2 Justificación de Relaciones Orientadas a Objetos

- Composición Estricta (Evaluation *-- Question): Cada reactivo de examen es generado al vuelo por el motor con permutaciones específicas para esa sesión. Las 10 preguntas carecen de valor ontológico o clave primaria natural fuera de la evaluación; si el examen se descarta o reintenta, sus 10 preguntas se destruyen en cascada en la memoria.
- Agregación (EvaluationController o-- StudentSession): La sesión del estudiante preexiste y sobrevive al examen individual. El controlador la referencia para actualizar rachas de estudio pero no controla exclusivamente su ciclo de vida.
- Realización (IEvaluationEngine <|.. DeterministicAIEngine): Implementación pura de contrato sin estado compartido.

CAPÍTULO IV: VISTA ESTRUCTURAL DE PERSISTENCIA

MULTIBASE DE DATOS

4.1 Resolución Estructural para Múltiples Bases de Datos (Diagrama 3)

En arquitecturas Offline-First coexisten múltiples motores de almacenamiento con distintos paradigmas: una base NoSQL local en el cliente (IndexedDB) optimizada para lecturas y escrituras atómicas en memoria flash, y una base documental remota en la nube (Cloud Firestore) con replicación multirregión. En UML, esta disparidad se resuelve separando las Entidades de Negocio, los Schemas Físicos de cada motor, los Data Mappers de transformación y el Repositorio Unificado.

Código Mermaid del Diagrama 3: Persistencia Multibase

```
classDiagram
    namespace Local_IndexedDB_Schemas {
        class EvaluationStoreRecord {
            +String evaluationId : keyPath
            +int levelId : index_by_level
            +String syncStatus : index_by_status
            +String questionsPayloadJSON
        }
        class SyncQueueRecord {
            +String queueId : keyPath
            +String operationType
            +int retryCount
        }
    }
    namespace Remote_Firestore_Schemas {
        class FirestoreEvaluationDocument {
            +String docId : path_evaluaciones_id
            +String studentUid : indexed_field
            +Timestamp createdAt : server_timestamp
        }
        class FirestoreQuestionSubdocument {
            +String questionId : path_preguntas_id
            +String statementText
        }
    }
    namespace Mappers_And_Repositories {
        class EvaluationLocalMapper {
            +toLocalRecord(domain: Evaluation) EvaluationStoreRecord
            +toDomain(record: EvaluationStoreRecord) Evaluation
        }
        class EvaluationRemoteMapper {
            +toFirestoreDocument(domain: Evaluation) FirestoreEvaluationDocument
        }
    }
```

```

        +toDomain(doc: FirestoreEvaluationDocument) Evaluation
    }
    class OfflineFirstEvaluationRepository {
        +saveEvaluation(eval: Evaluation) Promise~boolean~
        +synchronizePendingQueue() Promise~int~
    }
}
OfflineFirstEvaluationRepository --> EvaluationLocalMapper
OfflineFirstEvaluationRepository --> EvaluationRemoteMapper
FirestoreEvaluationDocument "1" *-- "10" FirestoreQuestionSubdocument

```

4.2 Justificación Técnica de Schemas Físicos y Mappers

- Schema IndexedDB (EvaluationStoreRecord): Emplea evaluationId como keyPath y define índices B-Tree (by_level, by_status) para filtrado instantáneo en el móvil. Las preguntas se serializan en un solo campo JSON (questionsPayloadJSON) para ejecutar una única transacción de I/O por evaluación, minimizando el desgaste de la memoria flash del smartphone.
- Schema Cloud Firestore (FirestoreEvaluationDocument y Subcolección): Se modela como documento padre en /evaluaciones/{id} con una subcolección anidada /preguntas/{qId}. Esto permite a los docentes ejecutar consultas analíticas sobre reactivos específicos sin descargar exámenes completos, optimizando la cuota de lecturas facturables de Firestore.
- Patrón Data Mapper: Desacopla la lógica de negocio de las APIs de Google y del navegador. La entidad Evaluation del dominio es completamente agnóstica de IndexedDB y Firestore.

CAPÍTULO V: MODELO CONCEPTUAL Y RELACIONAL DE DATOS (ERD)

5.1 Diagrama Entidad-Relación de 10 Entidades (Diagrama 4)

El Diagrama Entidad-Relación formaliza las 10 entidades del dominio de YAPU, estableciendo los tipos de datos normalizados, las restricciones de integridad referencial y las reglas de propagación de eliminaciones.

Código Mermaid del Diagrama 4: ERD

```
erDiagram
    USUARIO ||--o| PERFIL_ESTUDIANTE : "posee_perfil"
    USUARIO ||--o{ ORACION_BASE : "crea_como_autor"
    USUARIO ||--o{ ORACION_BASE : "certifica_como_validador"
    PERFIL_ESTUDIANTE ||--o{ PROGRESO_NIVEL : "registra_avance"
    NIVEL ||--o{ PROGRESO_NIVEL : "determina_nivel"
    NIVEL ||--o{ PALABRA_VOCABULARIO : "contiene_lexico"
    PERFIL_ESTUDIANTE ||--o{ VOCABULARIO_ESTUDIANTE : "gestiona_estudio"
    PALABRA_VOCABULARIO ||--o{ VOCABULARIO_ESTUDIANTE : "es_clasificada"
    NIVEL ||--o{ ORACION_BASE : "asocia_nivel"
    PERFIL_ESTUDIANTE ||--o{ EVALUACION : "rinde_evaluacion"
    NIVEL ||--o{ EVALUACION : "evalua_nivel"
    EVALUACION ||--|{ DETALLE_PREGUNTA : "genera_preguntas_composicion"
    ORACION_BASE ||--o{ DETALLE_PREGUNTA : "sirve_de_semilla"
    PERFIL_ESTUDIANTE ||--o{ RETO_COMUNITARIO : "propone_reto"
    USUARIO ||--o{ RETO_COMUNITARIO : "modera_reto"
```

5.2 Taxonomía de Relaciones de Persistencia

1. Relaciones Fuertes (Existencia Independiente): USUARIO a ORACION_BASE (asociación independiente de autor y validador) y NIVEL a PALABRA_VOCABULARIO (catálogo curricular). Si un docente es desactivado, las oraciones certificadas permanecen intactas.
2. Relaciones Débiles por Existencia: USUARIO a PERFIL_ESTUDIANTE (1:0..1). Si un estudiante solicita el borrado de su cuenta (Habeas Data boliviano), su perfil académico se purga en cascada (ON DELETE CASCADE).
3. Relaciones Débiles por Identificación: EVALUACION a DETALLE_PREGUNTA (composición pura 1:10) y PROGRESO_NIVEL (tabla asociativa con clave compuesta única id_estudiante + id_nivel).

CAPÍTULO VI: VISTA FUNCIONAL Y CASOS DE USO

6.1 Diagrama General de Casos de Uso (Diagrama 5)

Mapea los 10 Requisitos Funcionales del SRS frente a los 3 actores primarios: Estudiante, Docente Quechua y Administrador UPDS.

- Relaciones «include» Obligatorias: Registrar Cuenta incluye obligatoriamente Verificar Token Correo; Modo Offline incluye Sincronizar Progreso al Reconectar; Gestionar Oraciones y Proponer Retos incluyen Doble Validación Docente.
- Relación «extend» Condicional: Rendir Evaluación extiende a Desbloquear Siguiente Nivel si y solo si la calificación obtenida cumple la condición de guarda [calificación $\geq$ 70%].

6.2 Motor de IA Determinista vs. LLMs Comerciales (Diagrama 6.1)

El generador algorítmico local en TypeScript opera sin invocar APIs externas de OpenAI o Anthropic. Ejecuta un autómata que descompone oraciones base en componentes sintácticos (Sujeto, Objeto, Verbo), aplica sufijos aglutinantes controlados (-pi, -man, -manta) y extrae 3 distractores del mismo campo semántico mediante el algoritmo Fisher-Yates. Esto garantiza coste cero (\$0 en facturación mensual), respuesta en menos de 50 milisegundos y cero alucinaciones lingüísticas.

6.3 Protocolo de Doble Moderación Lingüística (Diagrama 6.2)

Aplica el principio de los cuatro ojos: ningún docente puede certificar sus propias oraciones base (`autor_id != validador_id`). Todo aporte comunitario entra en cuarentena en estado 'pendiente' hasta que un segundo docente hablante certificado valida la ortografía normalizada y la pertinencia etnográfica del contexto cultural.

CAPÍTULO VII: VISTA DINÁMICA Y DE COMPORTAMIENTO

7.1 Diagrama General de Secuencia Temporal (Diagrama 7)

Modela el ordenamiento temporal a lo largo de 5 fases críticas: (1) Carga inicial y validación de App Shell en Cache Storage; (2) Exploración curricular y estudio léxico; (3) Generación y corrección de evaluación determinista local; (4) Bifurcación de persistencia según conectividad (escritura inmediata en IndexedDB); y (5) Sincronización asíncrona en segundo plano (Background Sync) con vaciado de cola en lote (Batch Write) al reconectar a internet.

7.2 Máquina de Estados Finitos (Diagrama 8)

Modela los estados válidos de la sesión: VerificandoConectividad, DashboardActivo, NivelBloqueado (candado visual si nota previa < 70%), NivelHabilitado, EvaluacionDeterminista y PersistenciaProgreso (con estados EncoladoLocalIndexedDB y EsperandoReconexion).

7.3 Algoritmos de Actividad (Diagramas 9.1 y 9.2)

- Actividad A (Generación de Reactivos): Flujo con validación cruzada de categoría gramatical para distractores.
- Actividad B (Resiliencia Offline-First): Interceptación del Service Worker, almacenamiento no bloqueante en IndexedDB y vaciado transparente mediante SyncManager.

CAPÍTULO VIII: MATRIZ DE GOBERNANZA Y AUDITORÍA DE IA (UPDS)

En cumplimiento de las normativas éticas de la Universidad Privada Domingo Savio, esta sección documenta las intervenciones críticas donde el equipo humano modificó o descartó propuestas de IA generativa para garantizar la viabilidad del proyecto:

ID	Componente	Propuesta Asistida por IA	Ajuste Crítico Humano (Equipo UPDS)
AUD-01	Motor de IA (RF-005)	API comercial externa de OpenAI/Gemini	Descarte total. Motor determinista local en TypeScript (\$0 y offline).
AUD-02	Casos de Uso (RF-003)	Umbral del 50% o lectura lineal simple	Umbral estricto del 70% validado con docente titular.
AUD-03	Retos Comunitarios (RF-007)	Votación popular abierta tipo Reddit	Doble validación docente (4 ojos) con cuarentena obligatoria.
AUD-04	Distractores Léxicos	Selección aleatoria del diccionario global	Filtro semántico estricto: misma categoría gramatical y nivel.
AUD-05	Persistencia (ERD)	Relación simple Usuario 1:N OracionBase	Doble relación independiente: autor_id != validador_id.
AUD-06	Detalle de Preguntas	JSON no estructurado en tabla evaluación	Entidad débil DETALLE_PREGUNTA 1:10 normalizada para analítica.
AUD-07	Capa Presentación (RF-010)	SPA tradicional en React con Tailwind	Astro SSG + Islands. Consumo RAM < 150MB para gama de entrada.

AUD-08

Contexto Lingüístico

Solo palabra quechua
y traducción

Campo obligatorio
contexto_cultural
etnográfico andino.

CAPÍTULO IX: GUÍA DE EXPLICACIÓN PARA EVALUACIÓN ORAL (5 MINUTOS)

Para la defensa oral del trabajo grupal ante el tribunal académico de la UPDS (30 puntos asignados al criterio de verificación #2), el equipo ha distribuido los 300 segundos exactos conforme al siguiente cronograma técnico:

Intervalo	Módulo / Vista Expuesta	Estudiante Responsable
0:00 - 0:45	Contexto Territorial & Arquitectura General PWA de 4 Capas	Emmanuel Ponce Quiroga
0:45 - 2:00	Diagramas de Clases (Dominio & Multibase: IndexedDB y Firestore)	Jhoel Álvaro Cruz Zurita
2:00 - 3:15	Motor IA Determinista en TS & Doble Moderación Docente	Luis Mario Rocha Vela
3:15 - 4:15	Vista Dinámica: Secuencia, Estados & Background Sync	Jhoel Álvaro Cruz Zurita
4:15 - 5:00	Matriz de Gobernanza Ética de IA, Conclusiones & Q&A	Emmanuel Ponce Quiroga

9.1 Respuestas a Preguntas Críticas del Tribunal

• Pregunta: ¿Por qué en el diagrama de arquitectura no hay un rombo de decisión para comprobar si hay internet?

Respuesta: Porque la arquitectura modela topología estática y protocolos físicos de comunicación (HTTPS TLS 1.3, HTTP/2, Cache API). En una PWA, el cliente siempre emite llamadas HTTP estándar que son interceptadas de forma transparente por el Service Worker; modelar un rombo en la red es un error de abstracción conceptual que mezcla arquitectura con un diagrama de actividades.

• Pregunta: ¿Cómo se modela la coexistencia de múltiples bases de datos en UML?

Respuesta: Aplicando el patrón Data Mapper y Repository. Se definen las entidades de negocio en el Dominio, mientras que los schemas de IndexedDB (ObjectStores con keyPaths e índices) y Firestore (Colecciones con marcas de tiempo) se modelan en capas de persistencia independientes. Mappers específicos traducen entre el dominio y cada base de datos, y un repositorio unificado coordina la escritura local y la sincronización remota.

CONCLUSIONES

1. Cumplimiento Estricto del Gate 3: El Álbum de Modelos UML de YAPU proporciona una descripción completa, modular y ejecutable del sistema bajo el estándar IEEE Std 1016-2009, sentando las bases definitivas para la fase de construcción.
2. Resiliencia y Pertinencia Social: La combinación de Astro SSG, Service Workers e IndexedDB asegura que los estudiantes de comunidades rurales de Chuquisaca puedan aprender quechua en sus propios dispositivos móviles sin depender de conectividad constante ni consumir planes de datos.
3. Soberanía Pedagógica: El motor determinista en TypeScript y el principio de doble moderación garantizan rigor lingüístico y coste cero, protegiendo la integridad dialectal del quechua Collao.

REFERENCIAS BIBLIOGRÁFICAS (APA 7MA EDICIÓN)

- Cerrón-Palomino, R. (2003). *Lingüística quechua* (2.^a ed.). Centro de Estudios Regionales Andinos Bartolomé de Las Casas.
- Constitución Política del Estado Plurinacional de Bolivia. (2009). *Gaceta Oficial del Estado Plurinacional de Bolivia*.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7.^a ed.). Pearson.
- IEEE Computer Society. (2009). IEEE Std 1016-2009: IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE.
<https://doi.org/10.1109/IEEESTD.2009.5167255>
- IEEE Computer Society. (2011). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE. <https://doi.org/10.1109/IEEESTD.1998.88286>

Ministerio de Educación del Estado Plurinacional de Bolivia. (2023). Currículo Base del Sistema Educativo Plurinacional. La Paz, Bolivia.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software engineering (10.^a ed.). Pearson Education.

Wynne, M., & Hellesøy, A. (2017). The Cucumber book: Behaviour-driven development for testers and developers (2.^a ed.). Pragmatic Bookshelf.